EN3160 – Assignment 3: Neural Networks

Saruka U. | Index Number: 230585C
GitHub: https://github.com/Saruka03/EN3160-Assignment-03

Task 1 – Manually Implemented Dense Network

The CIFAR-10 network was modified to include a hidden layer with 100 neurons and sigmoid activation. Cross-entropy loss was used, and forward propagation, backpropagation, and parameter updates were implemented manually. The network was trained for 10 epochs.

Core architecture (PyTorch-style notation):

Listing 1: Manual forward and backward pass for Task 1

```
# Network: input -> 100 sigmoid units -> 10 classes
hidden = torch.sigmoid(x_train @ W1 + b1)
logits = hidden @ W2 + b2

# Cross-entropy from logits (labels contain class indices)
probs = torch.softmax(logits, dim=1)
loss = -torch.log_softmax(logits, dim=1)[
    torch.arange(len(labels)), labels
].mean()

# Manual gradients: softmax + cross-entropy
dlogits = probs.clone()
dlogits[torch.arange(len(labels)), labels] -= 1
dlogits /= len(labels)

dW2 = hidden.T @ dlogits
db2 = dlogits.sum(dim=0)

dhidden = dlogits @ W2.T
dhidden *= hidden * (1 - hidden)  # sigmoid derivative

dW1 = x_train.T @ dhidden
db1 = dhidden.sum(dim=0)

# Manual parameter updates
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
```

The training and test loss decreased throughout training. After 10 epochs, training and test accuracy reached 39.71%.

Table 1: Task 1 results after 10 epochs.

Metric	Accuracy
Training	39.71%
Test	39.71%

The similar training and test accuracies indicate comparable performance on both datasets. However, the overall accuracy remains modest, suggesting that the manually implemented dense network has limited classification capability for CIFAR-10 images.

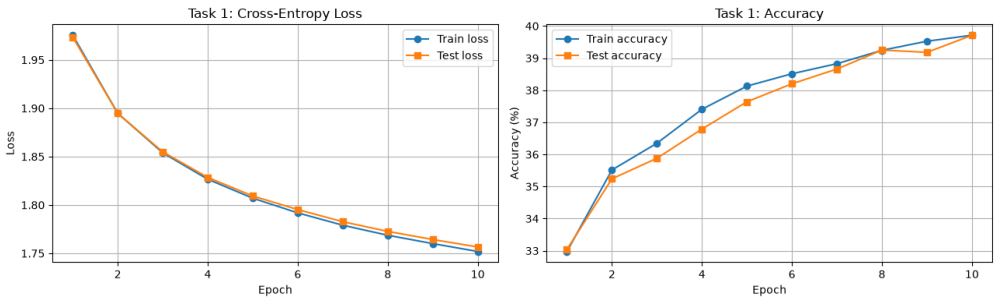


Figure 1: Training and test cross-entropy loss and accuracy over 10 epochs.

Task 2 – CNN Classification on MNIST

LeNet-5 and a modern three-layer CNN were trained on the MNIST dataset for 10 epochs using cross-entropy loss. The modern CNN incorporates batch normalization and dropout to improve training stability and reduce overfitting.

Listing 2: LeNet-5 architecture for MNIST

```
self.features = nn.Sequential(
    nn.Conv2d(1, 6, 5), nn.Tanh(),
    nn.AvgPool2d(2),
    nn.Conv2d(6, 16, 5), nn.Tanh(),
    nn.AvgPool2d(2)
)
self.classifier = nn.Sequential(
    nn.Linear(16*4*4, 120), nn.Tanh(),
    nn.Linear(120, 84), nn.Tanh(),
    nn.Linear(84, 10)
)
```

Listing 3: Modern CNN architecture for MNIST

```
nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1),
    nn.BatchNorm2d(32), nn.ReLU(),
    nn.MaxPool2d(2), nn.Dropout2d(0.25),
    nn.Conv2d(32, 64, 3, padding=1),
    nn.BatchNorm2d(64), nn.ReLU(),
    nn.MaxPool2d(2), nn.Dropout2d(0.25),
    nn.Conv2d(64, 128, 3, padding=1),
    nn.BatchNorm2d(128), nn.ReLU(),
    nn.MaxPool2d(2), nn.Dropout2d(0.25)
)
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(128*3*3, 128), nn.ReLU(),
    nn.Dropout(0.5), nn.Linear(128, 10)
)
```

Results after 10 Epochs

Table 2: Performance comparison of LeNet-5 and Modern CNN.

Model	Train Acc. (%)	Test Acc. (%)	Train Loss	Test Loss
LeNet-5	99.4017	98.6000	0.0184	0.0412
Modern CNN	99.6467	99.1700	0.0108	0.0272

The modern CNN achieved a test accuracy of 99.17%, outperforming LeNet-5 by 0.57 percentage points. It also achieved lower training and test losses. The training–test accuracy gap decreased from 0.80 percentage points for LeNet-5 to 0.48 percentage points for the modern CNN. These results indicate better classification performance on the MNIST test set.

Confusion Matrices

Figure 2 compares the per-epoch training and test accuracy and cross-entropy loss of both models over 10 epochs. Figure 3 presents the test-set confusion matrices for both models. These matrices show the distribution of correct classifications and misclassifications across the ten digit classes.

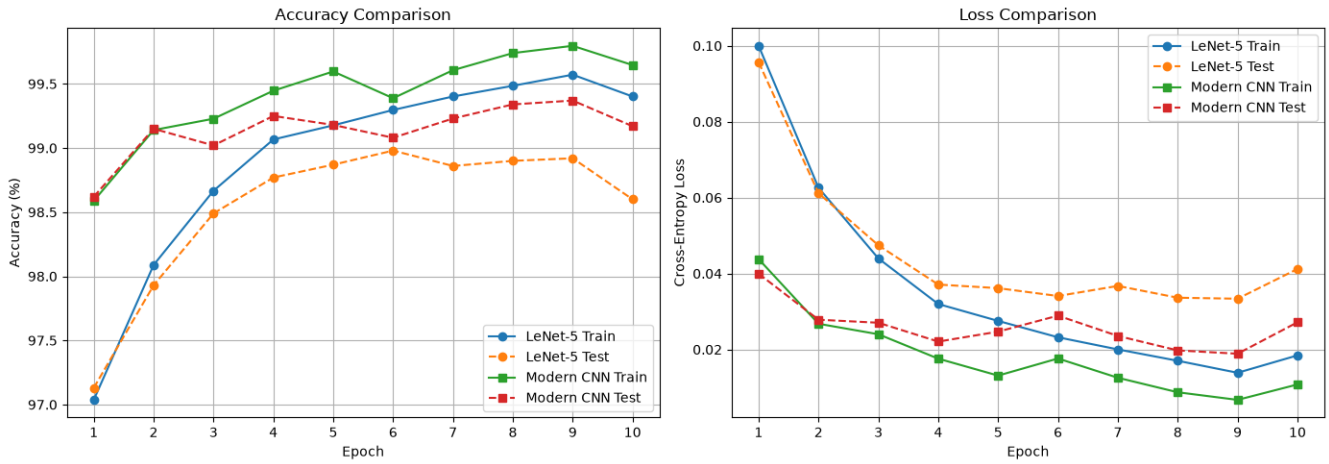


Figure 2: Per-epoch training and test accuracy and cross-entropy loss comparison between LeNet-5 and the Modern CNN.

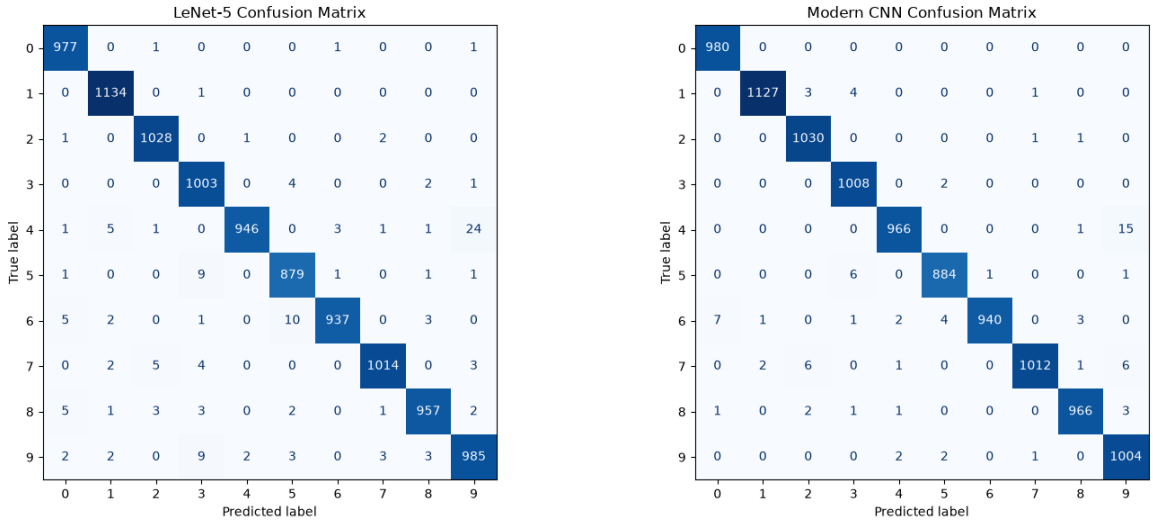


Figure 3: Test-set confusion matrices for LeNet-5 and the Modern CNN.

Task 3 – Transfer Learning with ResNet18

ImageNet-1K-pretrained ResNet18 was adapted to classify ants and bees. Images were resized to 224×224 and normalized using ImageNet statistics. For fine-tuning, all model parameters were trainable. For feature extraction, the pretrained backbone was frozen and only the final classifier was trained.

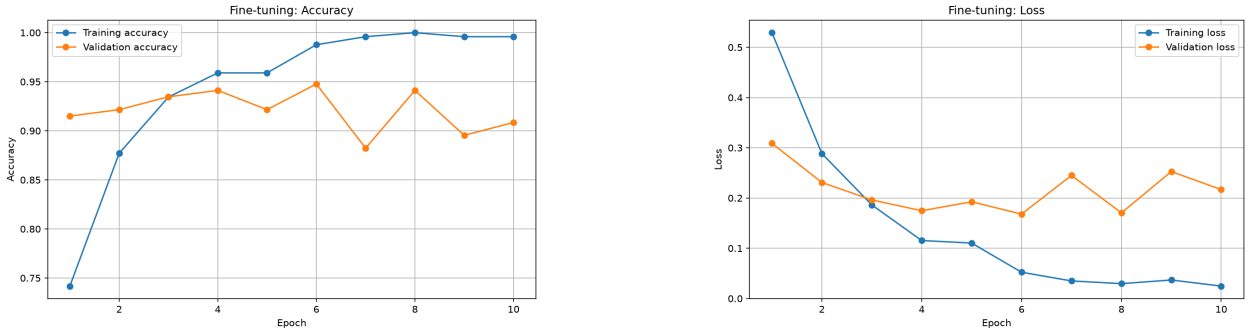
Key implementation:

```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False        # feature extraction
model.fc = nn.Linear(model.fc.in_features, 2)
optimizer = optim.SGD(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=0.001, momentum=0.9
)
```

Table 3: Recorded Hymenoptera results.

Method	Final train acc.	Best validation acc.
Fine-tuning	99.59%	94.77%
Feature extraction	94.26%	94.12%

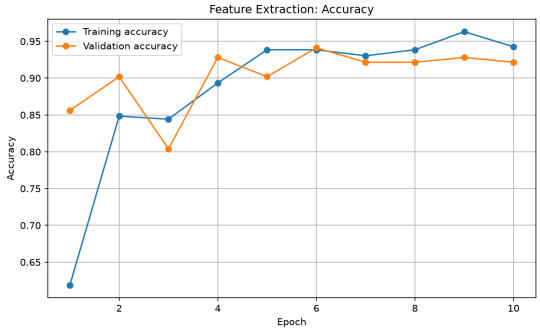
Fine-tuning had slightly higher best validation accuracy in this run. The difference is small, and the validation set is limited, so it should not be treated as evidence that fine-tuning will always be better.



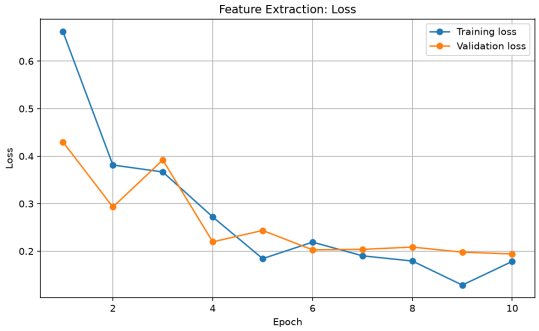
(a) ResNet18 fine-tuning accuracy.

(b) ResNet18 fine-tuning loss.

Figure 4: ResNet18 training results.



(c) ResNet18 feature-extraction accuracy.



(d) ResNet18 feature-extraction loss.

Figure 4: ResNet18 training results (continued).

Conclusion

The manually implemented CIFAR-10 network achieved 39.71% test accuracy. On MNIST, the modern CNN outperformed LeNet-5 with 99.17% test accuracy versus 98.60%. On Hymenoptera, ResNet18 fine-tuning achieved 94.77% best validation accuracy, while feature extraction achieved 94.12%. The modern CNN was the strongest of the tested MNIST models; both transfer-learning methods performed similarly on the small validation set.